Validation Report for the NS4KGE Knowledge Graph

Author: Ilyes Tebourski

Date: 20 July 2026

Scope: systematic, per-article validation of the extraction produced by the NS4KGE pipeline, over the **four entity types** of the KG: *datasets*, *metrics*, *KGE models*, *NS methods*.

Corpus: 55 scientific articles (negative sampling for KGE).

---

1. Objective

The NS4KGE knowledge graph is populated automatically by an LLM-based pipeline. The question addressed here is:

is this extraction reliable? For each entity type and each article, we measure:

- **precision**: is what the KG extracted actually present in the article?
- **(relative) recall**: is what is actually present in the article correctly extracted?

The validation produces both **quantitative metrics** and a **localized, verifiable list** of errors

(false positives) and likely omissions (false-negative candidates), directly usable to **correct the KG**.

---

2. Methodology (common to the four entity types)

2.1 Faithfulness to the pipeline

The pipeline applies **two prompts** to two preprocessed views of each article:

Prompt	Input (repository function)	What is extracted
`prompt1_no_results.txt`	`load_md_no_tables()` - article **without tables**	`proposedNSMethod`, `mentionedNSMethods`, `proposedKGEModel`, `mentionedKGEModels`...
`prompt2_tab.txt`	`load_md_tables_only()` - **tables only**	`Configurations[]`: `Dataset`, `Metric`, `KGEModel`, `NSMethod`

Key consequence: each entity is validated **against the exact input its prompt actually saw**.

A dataset (from tables) is searched in `tables_only`; a *mentioned* method (from prose) is searched in `no_tables`. The `tables_only`/`no_tables` views are **regenerated on the fly by calling the repository's own functions** (`ns4kge-extraction-pipeline`) from the source `.md` files in `liste_mds/`. There is therefore **no possible discrepancy** between what the validator sees and what the pipeline saw.

2.2 Fully deterministic validation (no LLM)

The evaluated KG was produced by an LLM; **the validation itself is a deterministic Python script**

(regular-expression matching). No API calls, no randomness: two runs yield **exactly the same result**.

The scores are thus not "judged by an AI" - they result from a mechanical, reproducible count.

2.3 Definitions

- **TP**: entity extracted by the KG **and** found in its source.
- **FP**: entity extracted but **absent** from its source -> precision error.
- **False-negative candidate**: entity present in the article but **not extracted**.
- **Precision** = TP / (TP + FP).

• **Relative recall** = TP / (TP + FN candidates). It is called *relative* because candidates are searched within a **global vocabulary** (the union of entities extracted across the 55 articles): for each article we test whether an entity known elsewhere appears in it without being extracted. This is a robust but **indicative** proxy (some candidates are mere passing mentions, to be validated manually).

2.4 Provenance distinction (for KGE models & NS methods)

These two entities come from **two sources**, which the ontology makes explicit:

- **Evaluated**: from `Configurations` (results tables) -> the model/method is **experimentally tested**.
- **Mentioned only**: from prose (`proposed`/`mentioned`) -> **cited** (related work...) without necessarily being evaluated.

We therefore report **4 scores** for each (precision x recall, evaluated x mentioned). This is an intrinsic value of the ontology: answering *"which models are actually compared in this article?"* vs *"which are merely cited?"* - impossible on raw text.

---

3. Results - the 12 scores

Entity	Prec. evaluated	Prec. mentioned	Recall evaluated	Recall mentioned
Datasets	98.9%	-	97.3%	-
Metrics	96.5%	-	96.0%	-
KGE Models	94.5%	97.9%	89.5%	95.5%
NS Methods	89.0%	87.0%	80.9%	71.2%

(Datasets and Metrics come only from tables -> 2 scores each; KGE Models and NS Methods -> 4 scores each.)

Overall reading: scores degrade steadily from top to bottom, and this is not accidental - it directly reflects the degree of standardization of the entity names (see §5).

---

4. Matching method, per entity

The principle is the same throughout (normalization + tolerant whole-word search), but the **matching difficulty increases** as name standardization decreases.

4.1 Datasets - strict verbatim matching

Canonical names (`FB15k-237`, `WN18RR`, `UMLS`, `YAGO3-10`). Case-insensitive matching, tolerant separators, **strict boundaries** so a prefix is not confused with a longer code (`FB15k` != `FB15k-237`). Recall restricted to `<table>` **cells** (a dataset must be a results row), with *stats* vs *results* table classification via the caption.

4.2 Metrics - alias-based matching

Metrics are **normalized by the prompt** (`Mean Rank`->`MR`, `Hit@10`->`Hits@10`, `Acc`->`Accuracy`). We therefore use **deterministic aliases** per canonical metric, and accept the generic header `Hits@N`/`Hits@k` (the table groups the N values) on the precision side.

4.3 KGE Models - verbatim + case + provenance

Standardized names (`TransE`, `DistMult`, `ComplEx`...). **Hybrid** matching: case-insensitive for precision (so as not to miss `ComplEx` vs `Complex`), **case-sensitive for recall** (names are stylized as words:

SimplE!="simple", RatE!="rate"). **Non-models** (GAN, MLP, BERT, word2vec… are not KGEModel - they are frameworks/components) are excluded from the recall vocabulary.

4.4 NS Methods - the hardest case (matching in 8 layers)

See §5 for the *why*. Matching required several successive layers:

1. **Exclusion of** Unknown (prompt placeholder for an unidentified baseline - 1634 occurrences).

2. **Canonical key** by content words: deduplicates near-duplicates (Bernoulli Sampling ≡ Bernoulli Negative Sampling).

3. **Qualifier normalization**: NoiGAN (hard) (40%) -> NoiGAN, NSCaching + scratch -> NSCaching, closed world constraints, neg_ratio=1 -> closed world constraints.

4. **Acronym derivation** (the main lever): Random Negative Sampling->RNS, Self-Adversarial NS->SANS, using initials of all words **and** of content words only.

5. **Search over the union of sources**: a *proposed* method is written Ours in the table but **spelled out in the prose** -> both must be searched.

6. **Position-based deduplication**: a text occurrence (e.g. SANS) is counted **once**, even if several vocabulary entries match it through acronym collision.

7. **Filtering of bare generic adjectives**: Random NS matched the word "random" in "Random Walk" -> filtered **in prose** (but kept in tables, where a Random/Uniform column is a real baseline).

8. **Exclusion of generic non-methods** (GAN).

---

5. Why the scores degrade (datasets -> NS methods)

This is the key point of the report. **The quality of the validation follows directly the degree of standardization of entity names**, which decreases sharply from one type to the next:

5.1 Datasets (98.9% / 97.3%) - universal, frozen names

KGE benchmarks are **highly standardized**: the whole community writes exactly FB15k-237, WN18RR, UMLS. Spelling variants are almost non-existent. Verbatim matching suffices -> near-perfect scores, **almost no preprocessing needed**. The only 2 FPs are extraction artifacts (Table 1 Dataset, FB13 (FB13_reduced)), not missed variants.

5.2 Metrics (96.5% / 96.0%) - small closed set, but LLM-normalized

There are only about **fifteen** metrics (MR, MRR, Hits@k, AUC…), i.e. a closed, well-known set. The only difficulty is that the prompt **rewrites** them (Mean Rank->MR, Hit@10->Hits@10), hence the alias layer. Notably, the 7 FPs are **genuine KG errors** (the Accuracy metric hallucinated in 5 articles where the word appears nowhere) - correctly caught by the validation.

5.3 KGE Models (94.5%-97.9% / 89.5%-95.5%) - proper, well-known, reused names

KGE models are **famous, published methods reused across articles**: TransE, DistMult, ComplEx, RotatE, ConvE… Their name is a **stable proper noun**, written the same way everywhere (up to case). An article using TransE writes TransE. Hence reliable matching and high scores. The only real difficulties are stylized casing (ComplEx, SimplE) and a few unsplit compounds (TransE+STC+TCE) that the KG should have separated - genuine defects, correctly flagged.

5.4 NS Methods (89.0%-87.0% / 80.9%-71.2%) - the weak link, by nature

This is where everything becomes harder, and it is **expected and explainable**:

• **Negative-sampling methods are NOT standardized.** Unlike KGE models, there is no shared catalogue of names. Each article invents/paraphrases its own method. Many do not even have an "official" name: in several cases, **the method name had to be constructed after the fact** (by the LLM or during annotation), from a description, because the article does not provide one.
-> *Typical example: where a model is simply called `TransE` everywhere, an NS method may be described as "we corrupt triples according to entity frequency" without ever being given a short name.*

• **Pervasive abbreviations.** The KG stores the full name (`Self-Adversarial Negative Sampling`) while the
article uses only the acronym (`SANS`, `Self-Adv`), or vice versa. Hence the need to **derive acronyms** - a step unnecessary for datasets/models.

• **"Negative Sampling" constantly omitted or abbreviated** (`Bernoulli Sampling` = `Bernoulli Negative Sampling`), and `**experimental qualifiers glued to the name**` (`NoiGAN (hard)` (40%), `TANS w/ Freq`) - hence
the normalization layers.

• **Inferred, unnamed baselines.** The LLM often attributes a standard baseline (`Uniform`, `Bernoulli`, `Random`, `MCMC`) to table rows **even though the article never names it explicitly**. This is not necessarily wrong - it is a plausible inference - but it is **impossible to prove by textual search**. These cases weigh on *measured* precision (counted as strict FPs) without necessarily being real errors. This is a **fundamental limitation** of any search-based validation, specific to NS methods.

• **Mention recall (71.2%) reflects a real KG behavior**: it misses ~1/3 of the methods cited in related work (`CAKE`, `IGAN`, `IRGAN`, `MixGCF`, `Gibbs NS`…). These are not spurious candidates - they are **genuine omissions**, hence an actionable result, not a measurement artifact.

In summary: the more an entity type is *named in a standardized way in the literature*, the easier the validation and the higher the score. Datasets and KGE models benefit from a frozen community nomenclature; NS methods, a younger and more fragmented area, lack this consensus - hence lower scores that **reflect the nature of the field as much as the quality of the extraction**.

---

6. Error analysis & honest choices

• **Precisions reported as strict values.** No entity was excluded from the denominator to inflate the score. In particular, **inferred baselines** (§5.4) are counted as FPs although they are not necessarily wrong - a deliberate choice, to hide nothing.

• **Legitimate matching fixes, distinguished from gaming.** Recall gains on NS methods come from fixing **counting bugs** (deduplicating a single occurrence by position; filtering a bare word like "random" that matched "Random Walk") - **not** from excluding entities. No genuine omission is hidden: real baseline columns remain counted.

• `Unknown` **excluded** from NS methods: it is a prompt placeholder, not a method.

• **Non-entities excluded from the vocabulary** (KGE: `GAN`/`MLP`/`BERT`; NS: `GAN`): these are not instances of `KGEModel`/`NSMethod`. Their presence in the vocabulary in fact revealed an **extraction inconsistency** on the KG side (the LLM sometimes classified them as models).

---

7. What the validation enables

Beyond the numbers, the output is **usable to correct the KG**:

• **False positives** -> entries to correct/remove (e.g. hallucinated `Accuracy`, `Table 1 Dataset`, unsplit compounds `TransE+STC+TCE`).

• **False-negative candidates** -> entities likely to add (table baselines, methods cited in related work not extracted).

• **Evaluated / mentioned distinction** validated in both directions (precision **and** recall), confirming that the KG correctly captures this ontology semantics.

The validation does not automatically correct the KG: it **localizes** errors verifiably, as a basis for targeted correction (manual or by re-extracting the affected articles).

---

8. Reproducibility

```
# from /home/ilyes/POSTER
python3 validation/validate_datasets.py      # datasets   -> reports_datasets/
python3 validation/validate_metrics.py       # metrics    -> reports_metrics/
python3 validation/validate_kgemodels.py     # KGE models -> reports_kgemodels/
python3 validation/validate_nsmethods.py     # NS methods -> reports_nsmethods/
```

Each script reads `liste_mds/*.md` and `ns4kge-kg/per_article/*_merged.json`, regenerates the `tables_only/`

`no_tables` views via the repository functions, and writes a detailed per-article report plus a `_SUMMARY.md`.

Appendix - raw figures per entity

Entity	Global vocab	Entities checked	FP	FN candidates
Datasets	73	184	2	5 (results) + 1 (stats)
Metrics	18	199	7	8
KGE Models	201	582 (344 eval + 238 ment)	~24	38 eval + 11 ment
NS Methods	154 (excl. `Unknown`)	331 (200 eval + 131 ment)	~43	42 eval + 46 ment